

Profiling and Optimizing LLM Inference at Scale

Saksham Rathi

Bachelor's Thesis Project - Stage 1
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents

1. Introduction
2. Background
3. Prior Work
4. Experiments - Single Family
5. Metrics
6. Experiments - Multiple Families
7. Future Steps

Motivation



- In many real-world settings, multiple users query the same underlying document or system prompt.
- For example, an Operating Systems teaching assistant bot may maintain a large shared context of lecture notes and examples, while each student asks short, context-specific questions.
- Similarly, enterprise assistants handle hundreds of independent queries over a shared knowledge base or product documentation.
- Recomputing attention over identical prefixes leads to redundant computation and wasted GPU cycles.
- Additionally, memory usage grows rapidly with batch size and KV cache accumulation.

Motivation



- In many real-world settings, multiple users query the same underlying document or system prompt.
- For example, an Operating Systems teaching assistant bot may maintain a large shared context of lecture notes and examples, while each student asks short, context-specific questions.
- Similarly, enterprise assistants handle hundreds of independent queries over a shared knowledge base or product documentation.
- Recomputing attention over identical prefixes leads to redundant computation and wasted GPU cycles.
- Additionally, memory usage grows rapidly with batch size and KV cache accumulation.

Motivation



- In many real-world settings, multiple users query the same underlying document or system prompt.
- For example, an Operating Systems teaching assistant bot may maintain a large shared context of lecture notes and examples, while each student asks short, context-specific questions.
- Similarly, enterprise assistants handle hundreds of independent queries over a shared knowledge base or product documentation.
- Recomputing attention over identical prefixes leads to redundant computation and wasted GPU cycles.
- Additionally, memory usage grows rapidly with batch size and KV cache accumulation.

Motivation



- In many real-world settings, multiple users query the same underlying document or system prompt.
- For example, an Operating Systems teaching assistant bot may maintain a large shared context of lecture notes and examples, while each student asks short, context-specific questions.
- Similarly, enterprise assistants handle hundreds of independent queries over a shared knowledge base or product documentation.
- Recomputing attention over identical prefixes leads to redundant computation and wasted GPU cycles.
- Additionally, memory usage grows rapidly with batch size and KV cache accumulation.

Motivation



- In many real-world settings, multiple users query the same underlying document or system prompt.
- For example, an Operating Systems teaching assistant bot may maintain a large shared context of lecture notes and examples, while each student asks short, context-specific questions.
- Similarly, enterprise assistants handle hundreds of independent queries over a shared knowledge base or product documentation.
- Recomputing attention over identical prefixes leads to redundant computation and wasted GPU cycles.
- Additionally, memory usage grows rapidly with batch size and KV cache accumulation.

Problem Statement



- Our goal is to characterize and analyze the performance of LLM inference workloads with shared prefixes.
- What are the key GPU-level metrics for evaluating such workloads?
- How do modern inference engines optimize for shared-prefix execution?
- How do mechanisms like prefix caching, chunked prefill, and request scheduling impact throughput and latency?
- What bottlenecks limit scalability as the number of concurrent prefix families increases?

Problem Statement



- Our goal is to characterize and analyze the performance of LLM inference workloads with shared prefixes.
- What are the key GPU-level metrics for evaluating such workloads?
- How do modern inference engines optimize for shared-prefix execution?
- How do mechanisms like prefix caching, chunked prefill, and request scheduling impact throughput and latency?
- What bottlenecks limit scalability as the number of concurrent prefix families increases?

Problem Statement



- Our goal is to characterize and analyze the performance of LLM inference workloads with shared prefixes.
- What are the key GPU-level metrics for evaluating such workloads?
- How do modern inference engines optimize for shared-prefix execution?
- How do mechanisms like prefix caching, chunked prefill, and request scheduling impact throughput and latency?
- What bottlenecks limit scalability as the number of concurrent prefix families increases?

Problem Statement



- Our goal is to characterize and analyze the performance of LLM inference workloads with shared prefixes.
- What are the key GPU-level metrics for evaluating such workloads?
- How do modern inference engines optimize for shared-prefix execution?
- How do mechanisms like prefix caching, chunked prefill, and request scheduling impact throughput and latency?
- What bottlenecks limit scalability as the number of concurrent prefix families increases?

Problem Statement



- Our goal is to characterize and analyze the performance of LLM inference workloads with shared prefixes.
- What are the key GPU-level metrics for evaluating such workloads?
- How do modern inference engines optimize for shared-prefix execution?
- How do mechanisms like prefix caching, chunked prefill, and request scheduling impact throughput and latency?
- What bottlenecks limit scalability as the number of concurrent prefix families increases?

Prefill vs Decode



- **Prefill:** Takes the whole user prompt as input and computes the probability of the first new token. Generate key and value vectors. Can be parallelized because all the tokens are known.
- **Decode:** Generates the remaining new tokens sequentially. Stops when the maximum length is reached, or when an end-of-sequence token is emitted. Autoregressive in nature.

Prefill vs Decode



- **Prefill:** Takes the whole user prompt as input and computes the probability of the first new token. Generate key and value vectors. Can be parallelized because all the tokens are known.
- **Decode:** Generates the remaining new tokens sequentially. Stops when the maximum length is reached, or when an end-of-sequence token is emitted. Autoregressive in nature.

Inference Engines

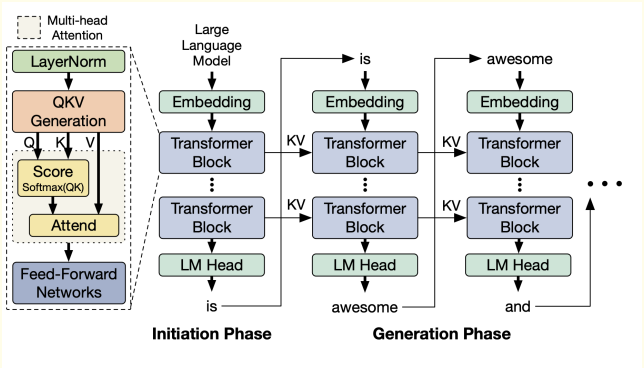


Figure: Architecture of a LLM

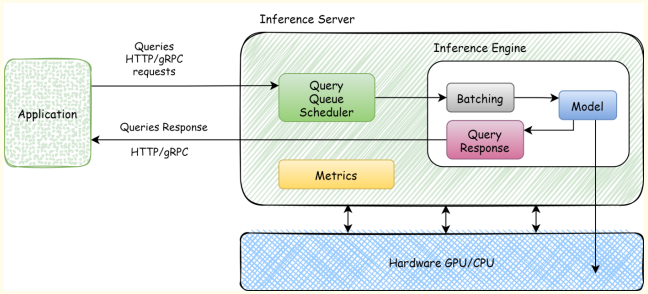


Figure: Architecture of an Inference Engine

- Systems layer that manages the execution of large language model (LLM) inference on hardware resources.
- Multiple user requests are grouped into a single batch to maximize GPU utilization and reduce per-request overhead.
- Memory Storage: Cache key-value pairs, reuse memory buffers, and apply quantization or weight-sharing.
- Offloading: Portions of the model state or intermediate activations can be offloaded to CPU when not needed actively.

Inference Engines

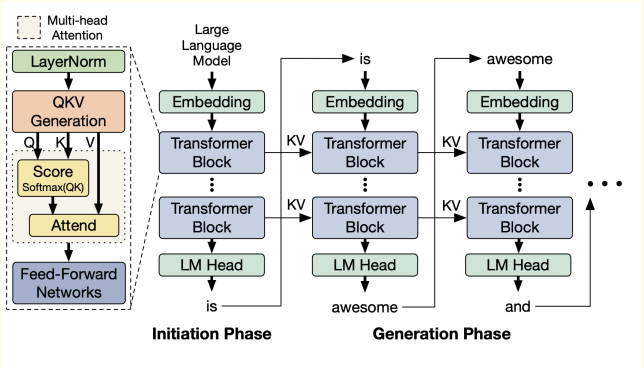


Figure: Architecture of a LLM

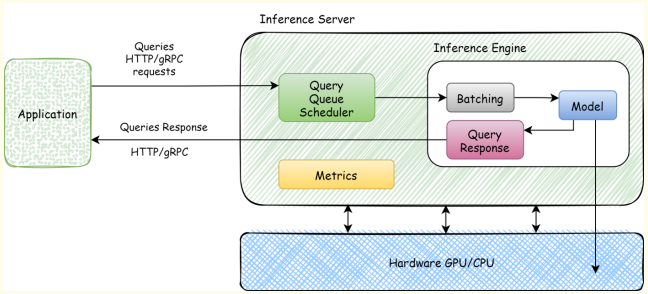


Figure: Architecture of an Inference Engine

- Systems layer that manages the execution of large language model (LLM) inference on hardware resources.
- Multiple user requests are grouped into a single batch to maximize GPU utilization and reduce per-request overhead.
- Memory Storage: Cache key-value pairs, reuse memory buffers, and apply quantization or weight-sharing.
- Offloading: Portions of the model state or intermediate activations can be offloaded to CPU when not needed actively.

Inference Engines

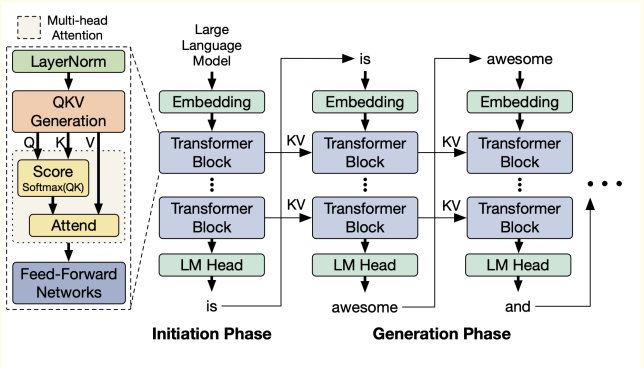


Figure: Architecture of a LLM

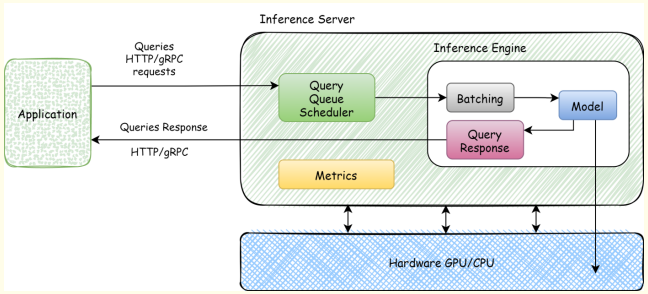


Figure: Architecture of an Inference Engine

- Systems layer that manages the execution of large language model (LLM) inference on hardware resources.
- Multiple user requests are grouped into a single batch to maximize GPU utilization and reduce per-request overhead.
- Memory Storage: Cache key-value pairs, reuse memory buffers, and apply quantization or weight-sharing.
- Offloading: Portions of the model state or intermediate activations can be offloaded to CPU when not needed actively.

Inference Engines

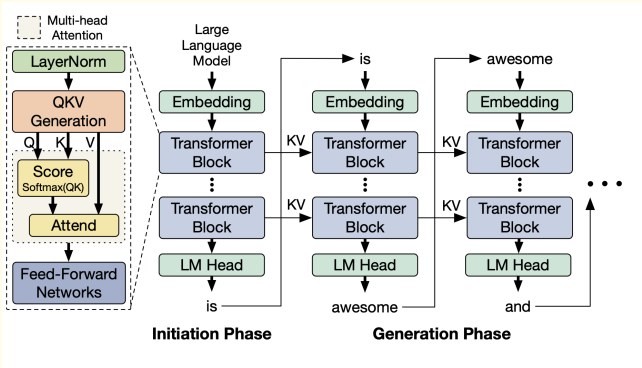


Figure: Architecture of a LLM

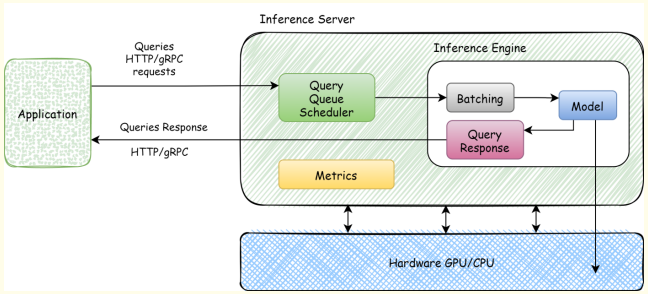


Figure: Architecture of an Inference Engine

- Systems layer that manages the execution of large language model (LLM) inference on hardware resources.
- Multiple user requests are grouped into a single batch to maximize GPU utilization and reduce per-request overhead.
- Memory Storage: Cache key-value pairs, reuse memory buffers, and apply quantization or weight-sharing.
- Offloading: Portions of the model state or intermediate activations can be offloaded to CPU when not needed actively.

vLLM



- vLLM adopts a centralized scheduler to coordinate the execution of distributed GPU workers.
- PagedAttention partitions the KV cache of each sequence into KV blocks. Removes internal and external fragmentation.
- Allows blocks to be shared when multiple samples share the same input prompt, or during beam search. Implements copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences.
- Evicted blocks are copied to the CPU memory.
- Each cached block is identified by the block hash - which comprises of the parent block hash, block tokens, and some extra fields.

vLLM



- vLLM adopts a centralized scheduler to coordinate the execution of distributed GPU workers.
- PagedAttention partitions the KV cache of each sequence into KV blocks. Removes internal and external fragmentation.
- Allows blocks to be shared when multiple samples share the same input prompt, or during beam search. Implements copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences.
- Evicted blocks are copied to the CPU memory.
- Each cached block is identified by the block hash - which comprises of the parent block hash, block tokens, and some extra fields.

vLLM



- vLLM adopts a centralized scheduler to coordinate the execution of distributed GPU workers.
- PagedAttention partitions the KV cache of each sequence into KV blocks. Removes internal and external fragmentation.
- Allows blocks to be shared when multiple samples share the same input prompt, or during beam search. Implements copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences.
- Evicted blocks are copied to the CPU memory.
- Each cached block is identified by the block hash - which comprises of the parent block hash, block tokens, and some extra fields.

vLLM



- vLLM adopts a centralized scheduler to coordinate the execution of distributed GPU workers.
- PagedAttention partitions the KV cache of each sequence into KV blocks. Removes internal and external fragmentation.
- Allows blocks to be shared when multiple samples share the same input prompt, or during beam search. Implements copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences.
- Evicted blocks are copied to the CPU memory.
- Each cached block is identified by the block hash - which comprises of the parent block hash, block tokens, and some extra fields.

vLLM



- vLLM adopts a centralized scheduler to coordinate the execution of distributed GPU workers.
- PagedAttention partitions the KV cache of each sequence into KV blocks. Removes internal and external fragmentation.
- Allows blocks to be shared when multiple samples share the same input prompt, or during beam search. Implements copy-on-write mechanism at the block granularity for the physical blocks that need modification by multiple sequences.
- Evicted blocks are copied to the CPU memory.
- Each cached block is identified by the block hash - which comprises of the parent block hash, block tokens, and some extra fields.

FastTree

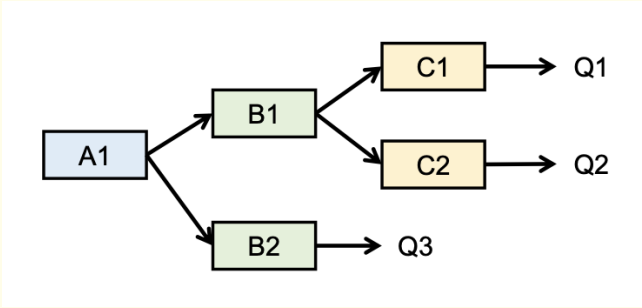


Figure: KV Cache Sharing through RadixTree

- Introduces GPU kernels specifically designed to process queries that share common prefixes within a radix tree structure.
- Gives an optimal edge assignment based on two types of losses - (1) Tile Padding (2) Maintaining Intermediate results
- Merges queries with the same prefix into a single kernel - (1) Decreases global memory transactions (2) Converts GEMV to GEMM operations

FastTree

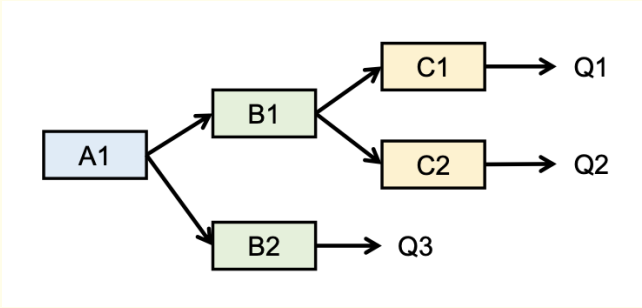


Figure: KV Cache Sharing through RadixTree

- Introduces GPU kernels specifically designed to process queries that share common prefixes within a radix tree structure.
- Gives an optimal edge assignment based on two types of losses - (1) Tile Padding (2) Maintaining Intermediate results
- Merges queries with the same prefix into a single kernel - (1) Decreases global memory transactions (2) Converts GEMV to GEMM operations

FastTree

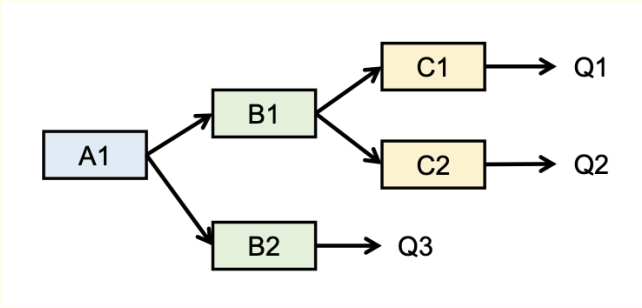


Figure: KV Cache Sharing through RadixTree

- Introduces GPU kernels specifically designed to process queries that share common prefixes within a radix tree structure.
- Gives an optimal edge assignment based on two types of losses - (1) Tile Padding (2) Maintaining Intermediate results
- Merges queries with the same prefix into a single kernel - (1) Decreases global memory transactions (2) Converts GEMV to GEMM operations

Workload Setup

- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Workload Setup

- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Workload Setup

- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Workload Setup

- **Goal:** Evaluate inference performance of LLMs under short queries with large shared context.
- **Models:** Qwen 0.5B, Tiny Llama 1.1B, Llama 3.1 8B, served using vLLM/SGLang/Transformers.
- **Hardware:** NVIDIA RTX6000 48GB GPU.
- **Metrics:** Throughput (req/s), latency, GPU utilization.

Parameter	Value/Range
System Prompt Length	≈1500 tokens (shared across requests)
Query Rate	5–700 req/sec
Query Length	10–20 tokens
Output Length	30–50 tokens
Batch Size	Adaptive

Table: Workload Characteristics

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Experiment Setup: Engine Comparison



- Compare inference engines: **Transformers**, **vLLM**, **SGLang**.
- Batch of 24 requests, each sharing the same system prompt.
- Each request: $\sim$20 input tokens $\rightarrow$ $\sim$40 generated tokens.
- Measure **total time taken** to process the batch.

Hypothesis

Engines leveraging prefix caching (e.g., SGLang) should perform far better than naive batching in Transformers.

Batch of requests through various Engines

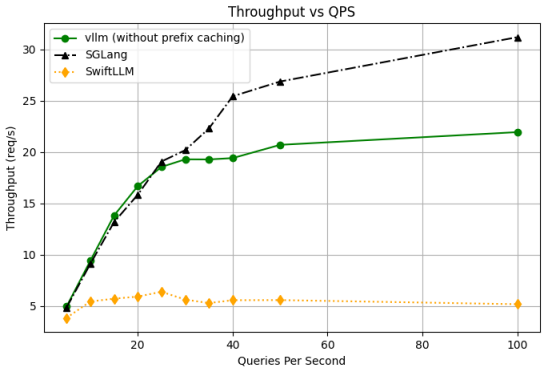


Figure: Throughput comparison across varying workloads

- **SwiftLLM:** lightweight vLLM clone (∼2000 LOC), but doesn't performs that well.
- **SGLang:** achieves best performance via radix-tree prefix sharing.

Technique Used	Time (s)
Transformers (batch = 1)	30.21
Transformers (batch = 2)	19.51
Transformers (batch = 3)	18.64
vLLM (no system prompt pre-filling)	0.45
SGLang (fork)	1.03
SGLang (run_batch)	0.40

Table: Measuring the time taken to process a batch of 24 requests using different techniques

Experiment Setup: Request Timelines



- Aim: Measure time spent in **prefill** vs **decode** stages for 20 req/sec workload (sending a new request after every 0.05 seconds).
- Using vLLM V0 engine without chunked prefill.
- Tool: Custom logging of per-request timestamps.

Figure: Simplified per-request timeline

Experiment Setup: Request Timelines



- Aim: Measure time spent in **prefill** vs **decode** stages for 20 req/sec workload (sending a new request after every 0.05 seconds).
- Using vLLM V0 engine without chunked prefill.
- Tool: Custom logging of per-request timestamps.

Figure: Simplified per-request timeline

Experiment Setup: Request Timelines



- Aim: Measure time spent in **prefill** vs **decode** stages for 20 req/sec workload (sending a new request after every 0.05 seconds).
- Using vLLM V0 engine without chunked prefill.
- Tool: Custom logging of per-request timestamps.

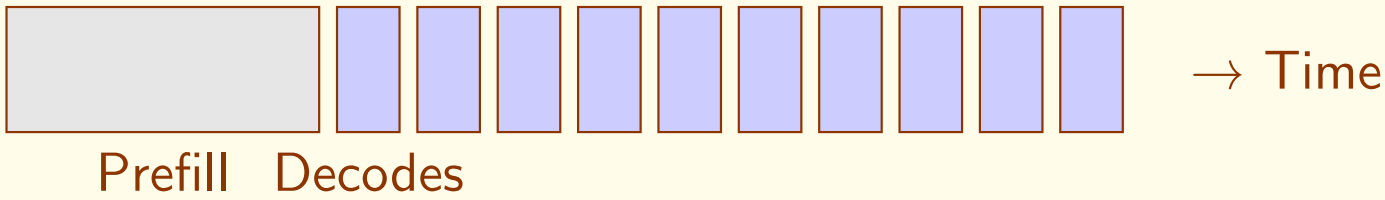


Figure: Simplified per-request timeline

Results: LLM Request Timelines

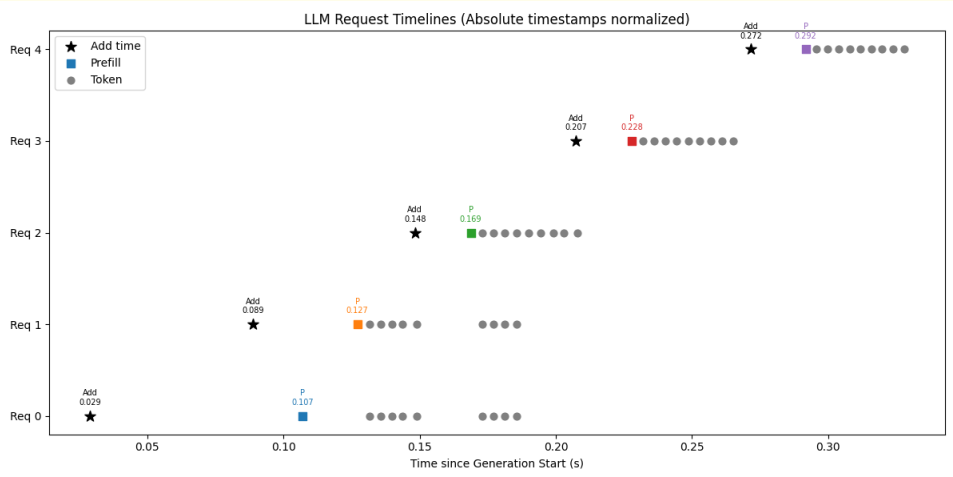


Figure: 20 Requests/sec – No chunked prefill – vLLM V0

- Prefill dominates runtime due to large prompt.
- Prefill and decode cannot overlap here.
- Low TTFT but high total latency.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: LLM Request Timelines

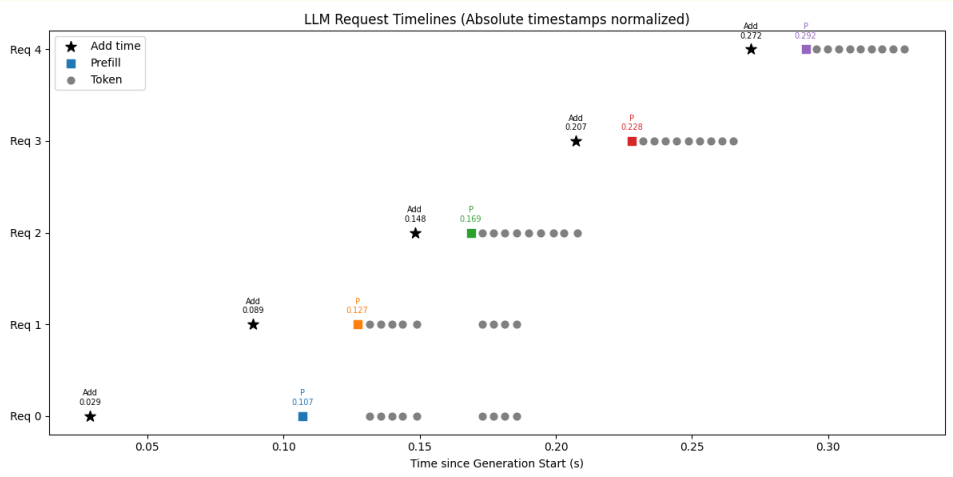


Figure: 20 Requests/sec – No chunked prefill – vLLM V0

- Prefill dominates runtime due to large prompt.
- Prefill and decode cannot overlap here.
- Low TTFT but high total latency.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: LLM Request Timelines

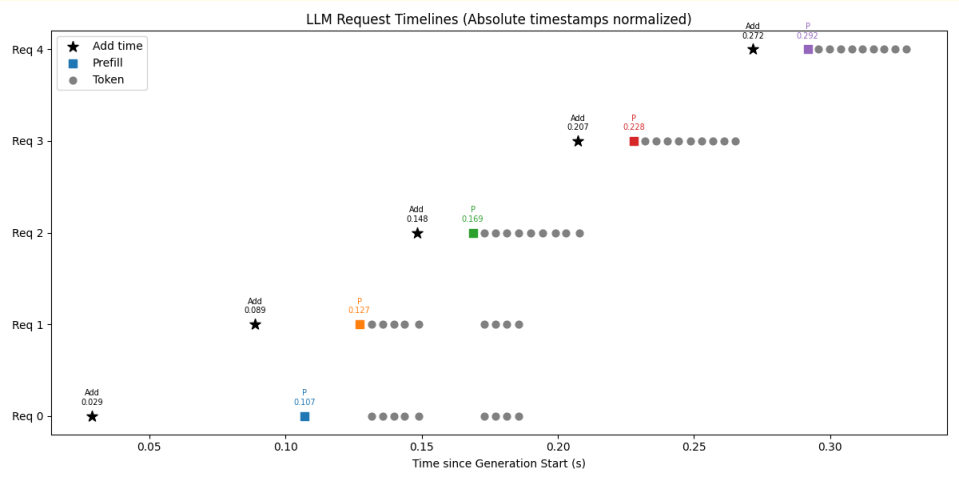


Figure: 20 Requests/sec – No chunked prefill – vLLM V0

- Prefill dominates runtime due to large prompt.
- Prefill and decode cannot overlap here.
- Low TTFT but high total latency.

Results: Throughput vs Load for vLLM

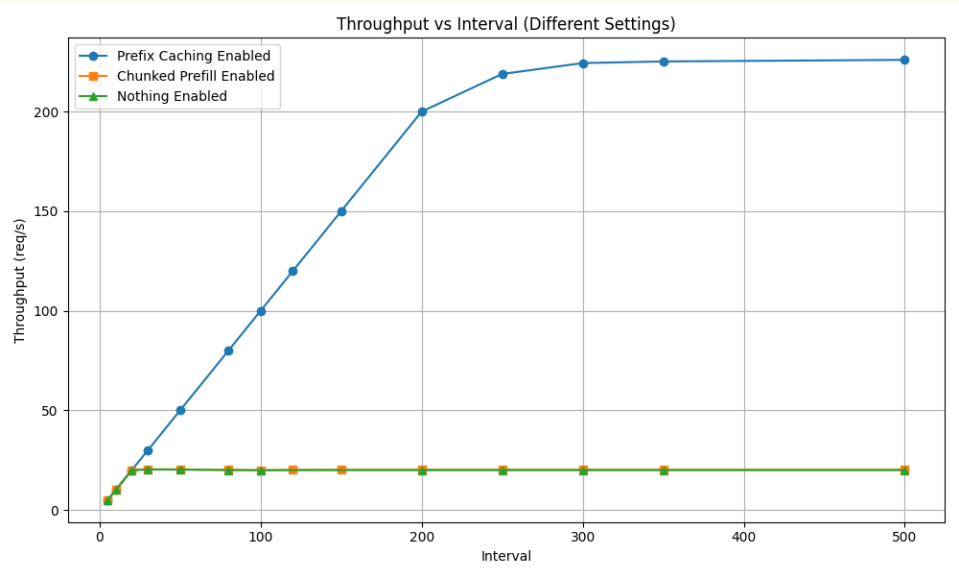


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

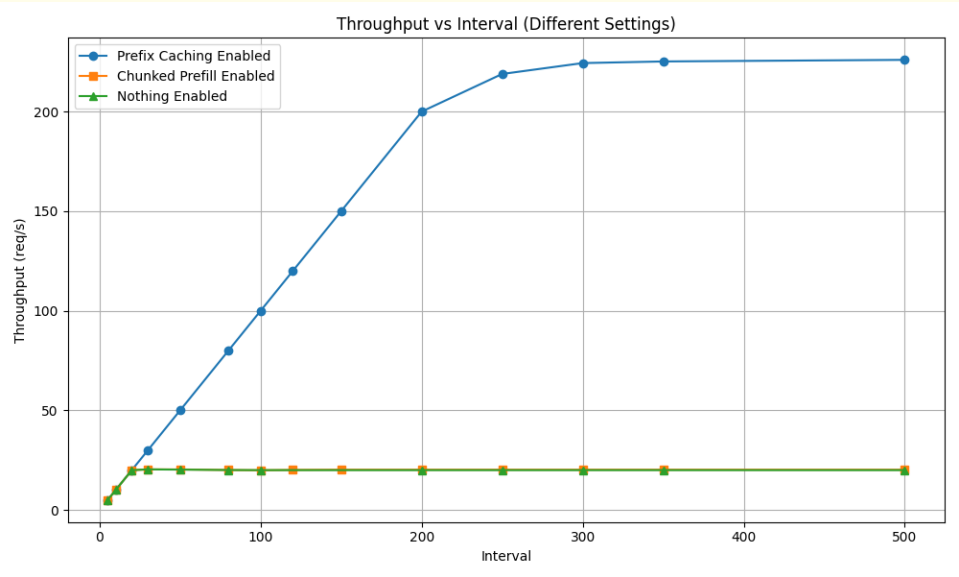


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

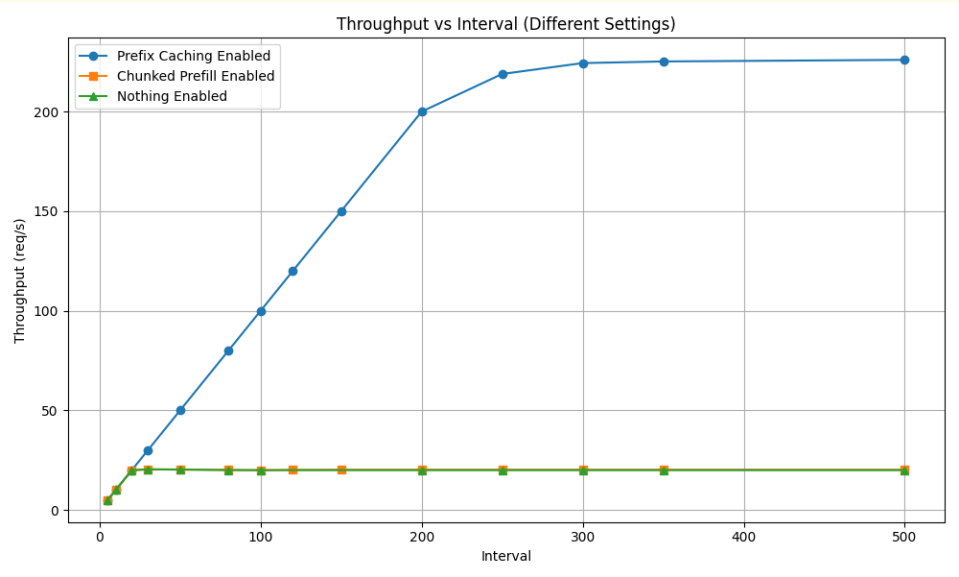


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

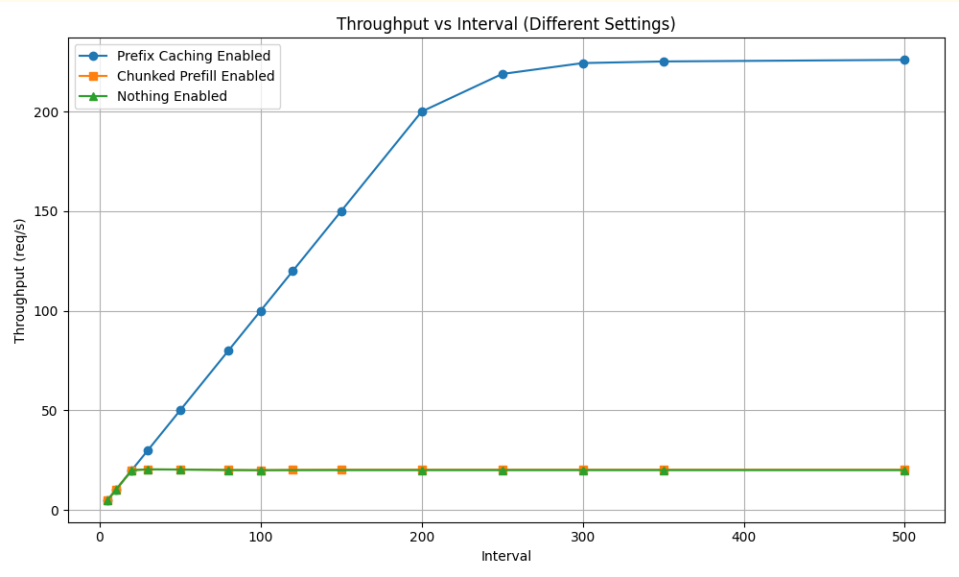


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Results: Throughput vs Load for vLLM

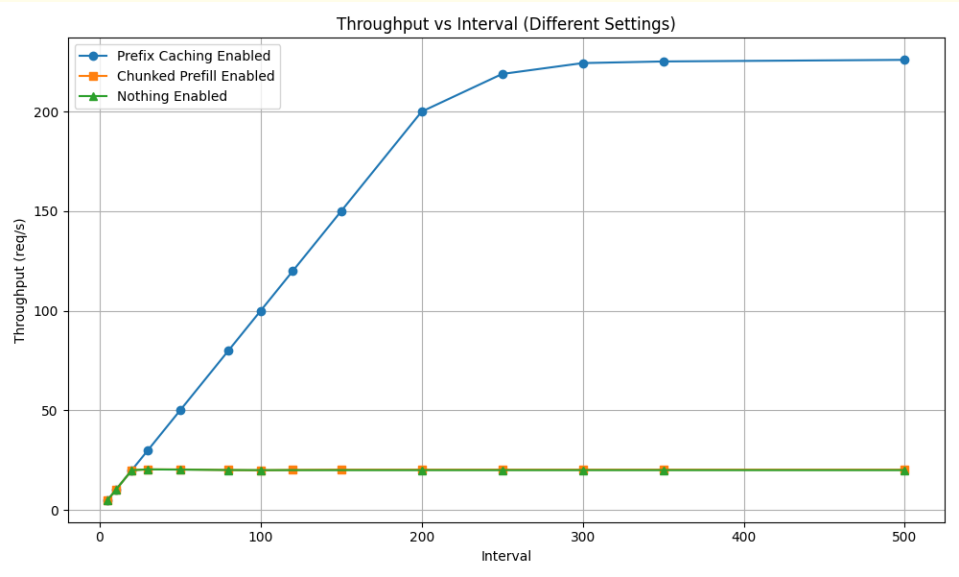


Figure: Throughput vs Load

- Varying the input load, and comparing different modes.
- Chunked prefill: limited benefit for small queries.
- Prefix caching: major boost to throughput.
- Batch tuning further improves efficiency.

Observation

Saturation begins at ∼300 req/sec — compute bottleneck.

Key GPU Metrics



- We employ two primary NVIDIA tools for this study: Nsight Systems (**NSYS**) and the Data Center GPU Management Interface (**DCGMI**).
- **SM Activity:** The fraction of time at least one warp was active on a multiprocessor, averaged over all multiprocessors.
- **SM Occupancy:** The fraction of resident warps on a multiprocessor, relative to the maximum number of concurrent warps supported on a multiprocessor.
- **Tensor Core Utilization:** The fraction of cycles the tensor pipe was active.

Key GPU Metrics



- We employ two primary NVIDIA tools for this study: Nsight Systems (**NSYS**) and the Data Center GPU Management Interface (**DCGMI**).
- **SM Activity:** The fraction of time at least one warp was active on a multiprocessor, averaged over all multiprocessors.
- **SM Occupancy:** The fraction of resident warps on a multiprocessor, relative to the maximum number of concurrent warps supported on a multiprocessor.
- **Tensor Core Utilization:** The fraction of cycles the tensor pipe was active.

Key GPU Metrics



- We employ two primary NVIDIA tools for this study: Nsight Systems (**NSYS**) and the Data Center GPU Management Interface (**DCGMI**).
- **SM Activity:** The fraction of time at least one warp was active on a multiprocessor, averaged over all multiprocessors.
- **SM Occupancy:** The fraction of resident warps on a multiprocessor, relative to the maximum number of concurrent warps supported on a multiprocessor.
- **Tensor Core Utilization:** The fraction of cycles the tensor pipe was active.

Key GPU Metrics



- We employ two primary NVIDIA tools for this study: Nsight Systems (**NSYS**) and the Data Center GPU Management Interface (**DCGMI**).
- **SM Activity:** The fraction of time at least one warp was active on a multiprocessor, averaged over all multiprocessors.
- **SM Occupancy:** The fraction of resident warps on a multiprocessor, relative to the maximum number of concurrent warps supported on a multiprocessor.
- **Tensor Core Utilization:** The fraction of cycles the tensor pipe was active.

NSYS Run on Matrix Multiplication

- Various Phases: Host to device copy, kernel computation, and device to host copy.
- Compute metrics reach high values during kernel execution phase.

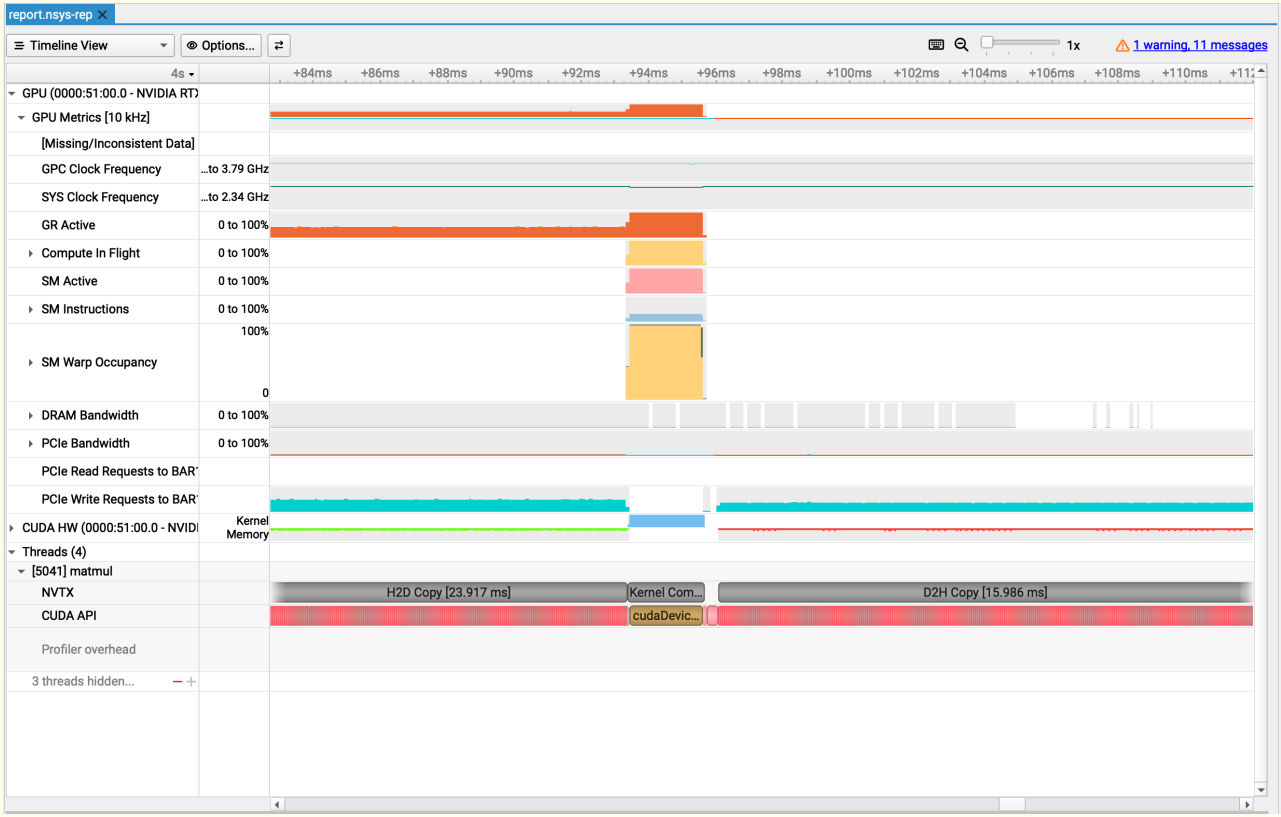


Figure: NSYS Metrics Snapshot

Multiple Families - Generation of such a workload

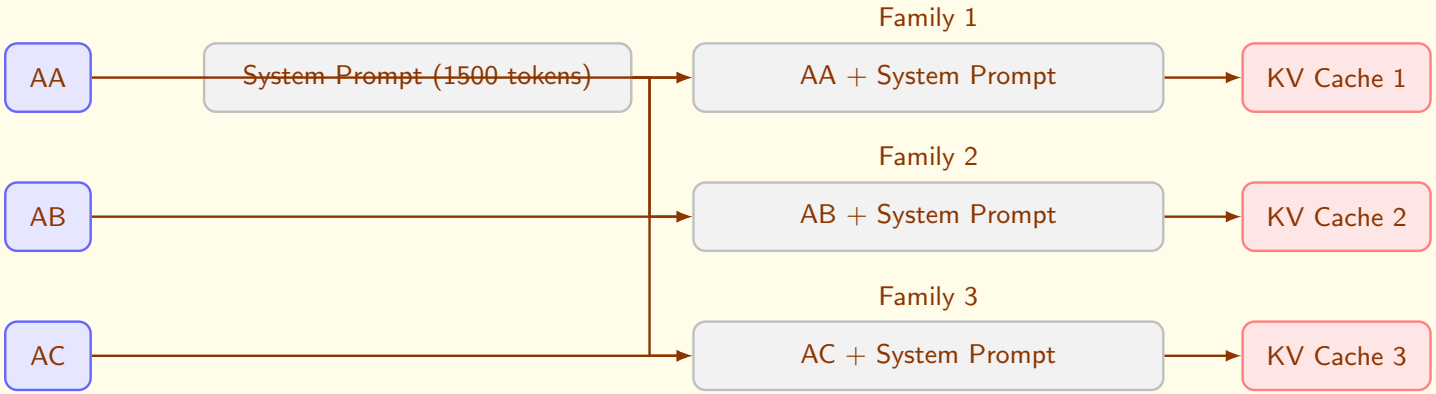


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes → separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Multiple Families - Generation of such a workload

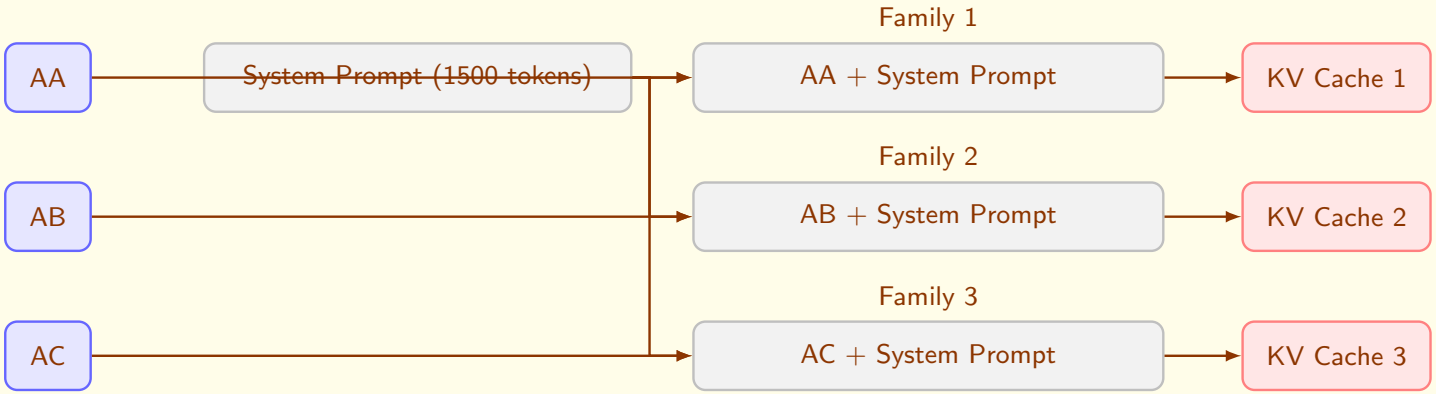


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes → separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Multiple Families - Generation of such a workload

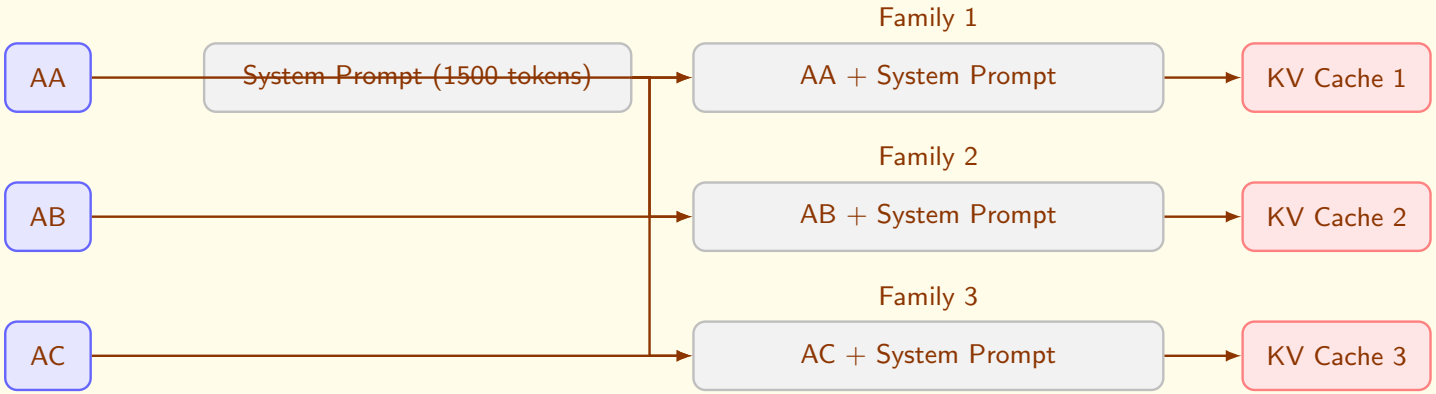


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes → separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Multiple Families - Generation of such a workload

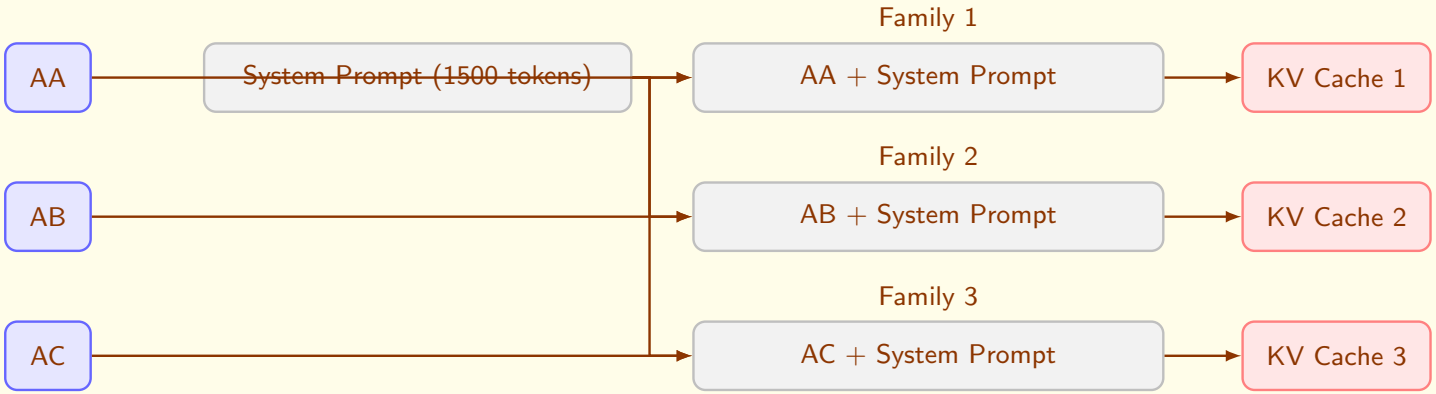


Figure: Multiple Families' Workload Generation

- What we have already - a large system prompt of around 1500 tokens.
- A Naive Way - Write down multiple such system prompts, and 100 queries based on each one of them.
- A Better Way - Add unique identifiers ("AA", "AB") to the existing system prompt.
- Different block hashes $\rightarrow$ separate KV Cache blocks for each family.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Effect of different Models

Model Chosen	10 families, each with 1000 queries	1 family with 10000 requests
Qwen 0.5B	294.228 req/s	308.750 req/s
Llama 7B	43.923 req/s	124.347 req/s
LongChat 13B	22.415 req/s	70.097 req/s

Table: Throughput with multiple families across various models

- As the model size increases, the throughput degradation becomes more pronounced.
- Primarily because of - (1) Increased attention computation overhead (2) Increase in the working set size - more transactions with global memory.

Effect of different Models

Model Chosen	10 families, each with 1000 queries	1 family with 10000 requests
Qwen 0.5B	294.228 req/s	308.750 req/s
Llama 7B	43.923 req/s	124.347 req/s
LongChat 13B	22.415 req/s	70.097 req/s

Table: Throughput with multiple families across various models

- As the model size increases, the throughput degradation becomes more pronounced.
- Primarily because of - (1) Increased attention computation overhead (2) Increase in the working set size - more transactions with global memory.

Same Family → Single Batch

Number of Families	Number of queries in each family	Max Batch Size	Ordered by Family ID	Random Permutation
10	10	10	12.28	11.981
10	100	100	59.601	55.704
10	100	200	61.693	60.923
100	100	100	42.882	39.172

Table: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

- An insight: merging queries from the same family into a single batch leads to significant throughput improvements.
- Primarily because of limited L2 cache size - can't fit KV Caches of multiple families. Leads to more HBM loads and lower throughput.
- Merging same family queries into a single batch effectively reduces the unique KV Cache blocks needed for that batch.

Same Family → Single Batch

Number of Families	Number of queries in each family	Max Batch Size	Ordered by Family ID	Random Permutation
10	10	10	12.28	11.981
10	100	100	59.601	55.704
10	100	200	61.693	60.923
100	100	100	42.882	39.172

Table: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

- An insight: merging queries from the same family into a single batch leads to significant throughput improvements.
- Primarily because of limited L2 cache size - can't fit KV Caches of multiple families. Leads to more HBM loads and lower throughput.
- Merging same family queries into a single batch effectively reduces the unique KV Cache blocks needed for that batch.

Same Family → Single Batch



Number of Families	Number of queries in each family	Max Batch Size	Ordered by Family ID	Random Permutation
10	10	10	12.28	11.981
10	100	100	59.601	55.704
10	100	200	61.693	60.923
100	100	100	42.882	39.172

Table: Comparison of Throughput (req/sec) when merging queries from the same family into a single batch vs not merging

- An insight: merging queries from the same family into a single batch leads to significant throughput improvements.
- Primarily because of limited L2 cache size - can't fit KV Caches of multiple families. Leads to more HBM loads and lower throughput.
- Merging same family queries into a single batch effectively reduces the unique KV Cache blocks needed for that batch.

Prior Prompt Prefilling - to compare decode performance

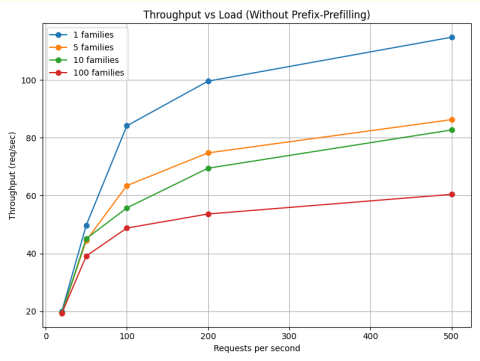


Figure: Without prefix pre-filling

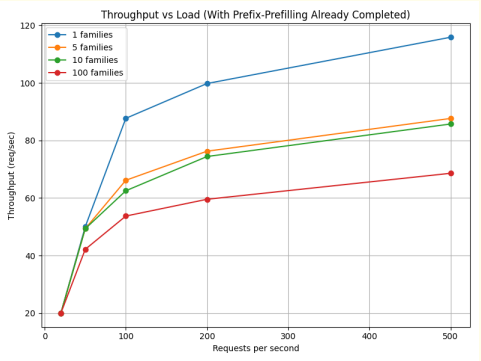


Figure: With prefix pre-filling

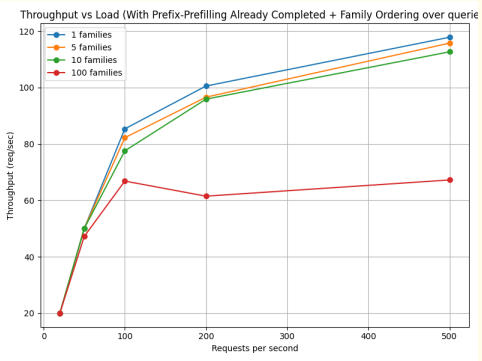


Figure: Ordered by family IDs

- The second image - all system prompts passed in the model beforehand - KV Caches already residing in the HBM. But still, a significant difference between the throughput numbers across different number of families.
- Computation of KV Cache Blocks - not the main bottleneck.
- Rather, the memory bandwidth to load these blocks from HBM to L2 cache.
- Ordering the requests by family ID → better cache locality, reduced cache thrashing.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Prior Prompt Prefilling - to compare decode performance

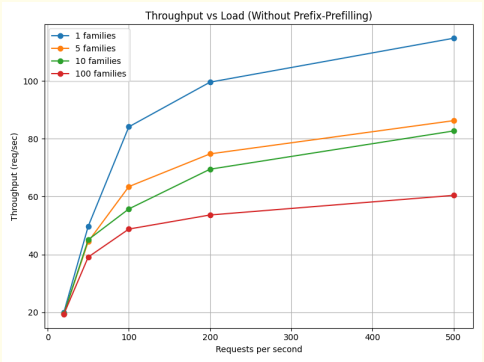


Figure: Without prefix pre-filling

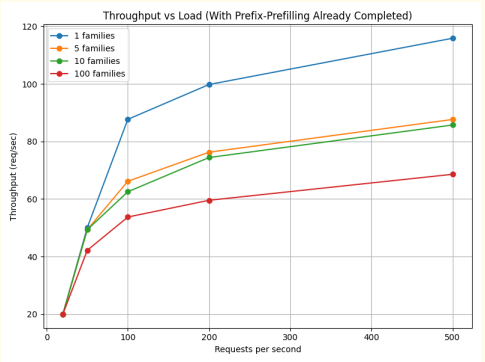


Figure: With prefix pre-filling

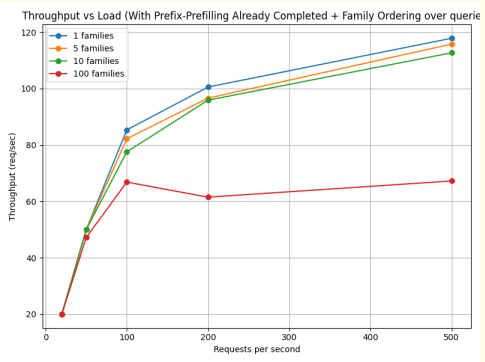


Figure: Ordered by family IDs

- The second image - all system prompts passed in the model beforehand - KV Caches already residing in the HBM. But still, a significant difference between the throughput numbers across different number of families.
- Computation of KV Cache Blocks - not the main bottleneck.
- Rather, the memory bandwidth to load these blocks from HBM to L2 cache.
- Ordering the requests by family ID → better cache locality, reduced cache thrashing.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Prior Prompt Prefilling - to compare decode performance

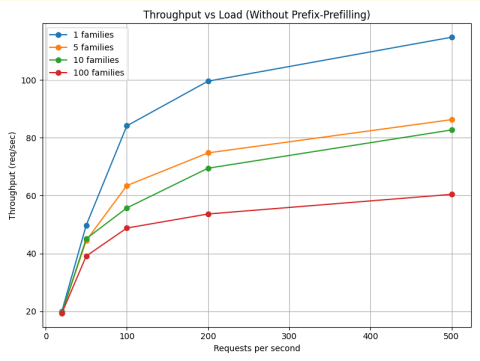


Figure: Without prefix pre-filling

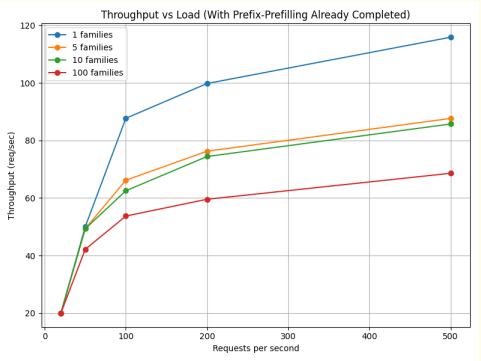


Figure: With prefix pre-filling

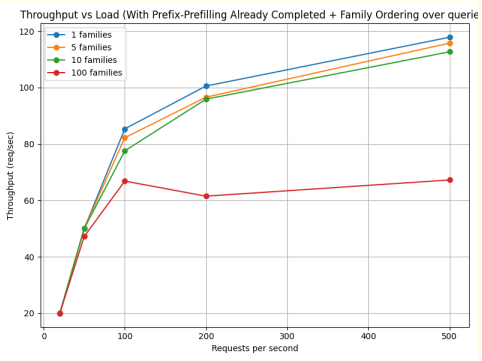


Figure: Ordered by family IDs

- The second image - all system prompts passed in the model beforehand - KV Caches already residing in the HBM. But still, a significant difference between the throughput numbers across different number of families.
- Computation of KV Cache Blocks - not the main bottleneck.
- Rather, the memory bandwidth to load these blocks from HBM to L2 cache.
- Ordering the requests by family ID → better cache locality, reduced cache thrashing.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Prior Prompt Prefilling - to compare decode performance

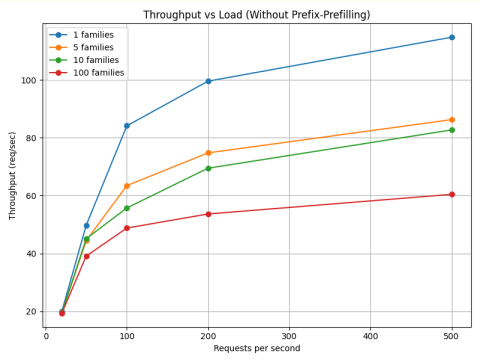


Figure: Without prefix pre-filling

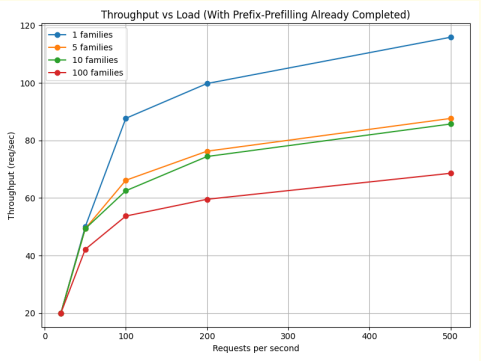


Figure: With prefix pre-filling

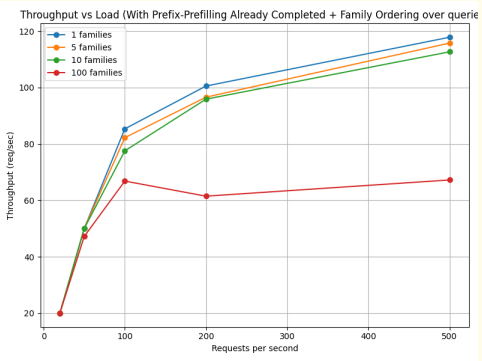
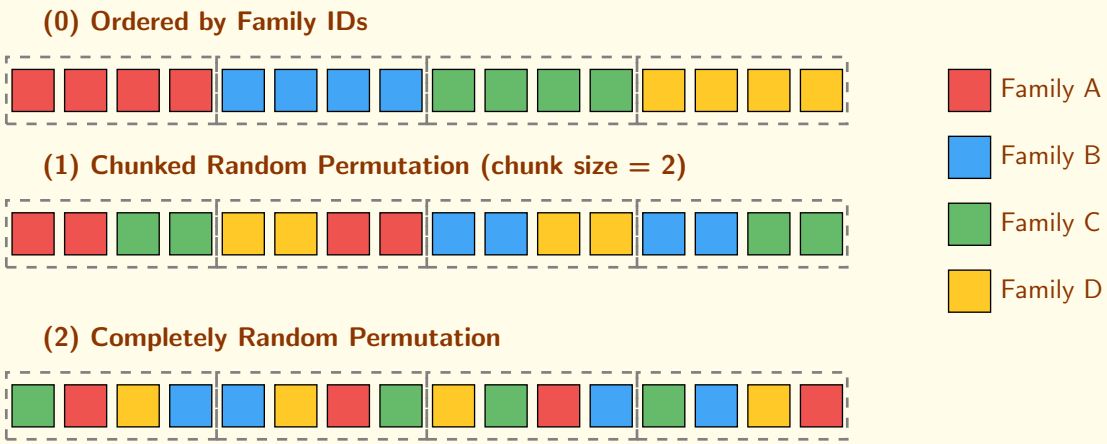


Figure: Ordered by family IDs

- The second image - all system prompts passed in the model beforehand - KV Caches already residing in the HBM. But still, a significant difference between the throughput numbers across different number of families.
- Computation of KV Cache Blocks - not the main bottleneck.
- Rather, the memory bandwidth to load these blocks from HBM to L2 cache.
- Ordering the requests by family ID $\rightarrow$ better cache locality, reduced cache thrashing.

Varying Multiple Parameters Together

- Number of Families
- Workload - requests per second
- Max Batch Size
- The nature of the batch - whether ordered by family IDs or random permutation:
  - 0: Ordered by family IDs
  - 1: We make chunks of size 50 from the sorted list and permute each chunk randomly (allows us to have some level of prefix sharing between consecutive queries)
  - 2: Completely random permutation



Varying Multiple Parameters Together

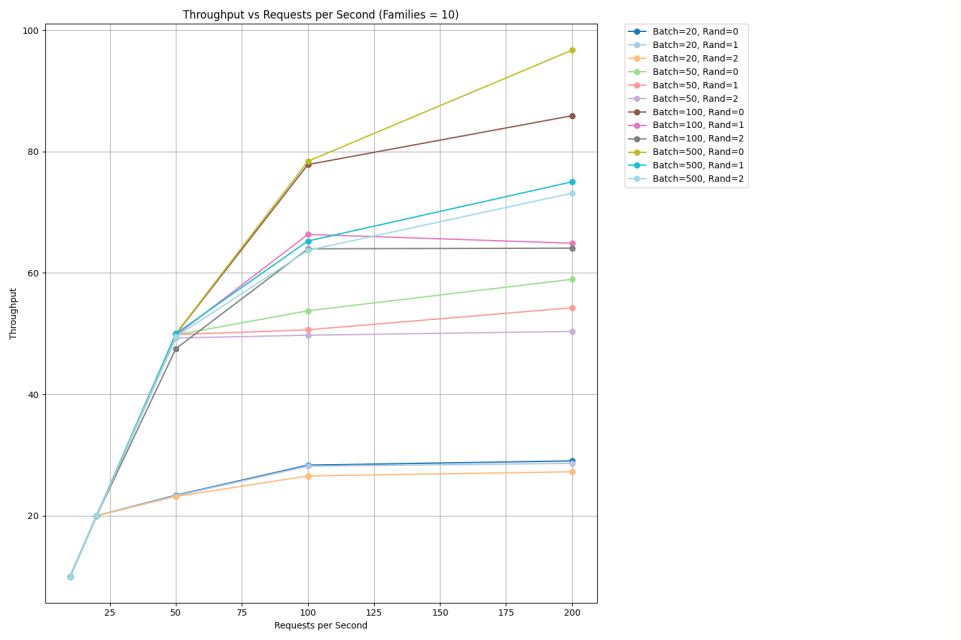


Figure: Throughput vs Workload by varying other parameters (10 families)

- Increase in batch size $\rightarrow$ higher throughput.
- A batch nature level of 1 is roughly similar to level 2 $\rightarrow$ even a small amount of mixing leads to significant degradation.
- A homogeneous batch of smaller size at times performs siginificantly better than a larger heterogeneous batch.

Varying Multiple Parameters Together

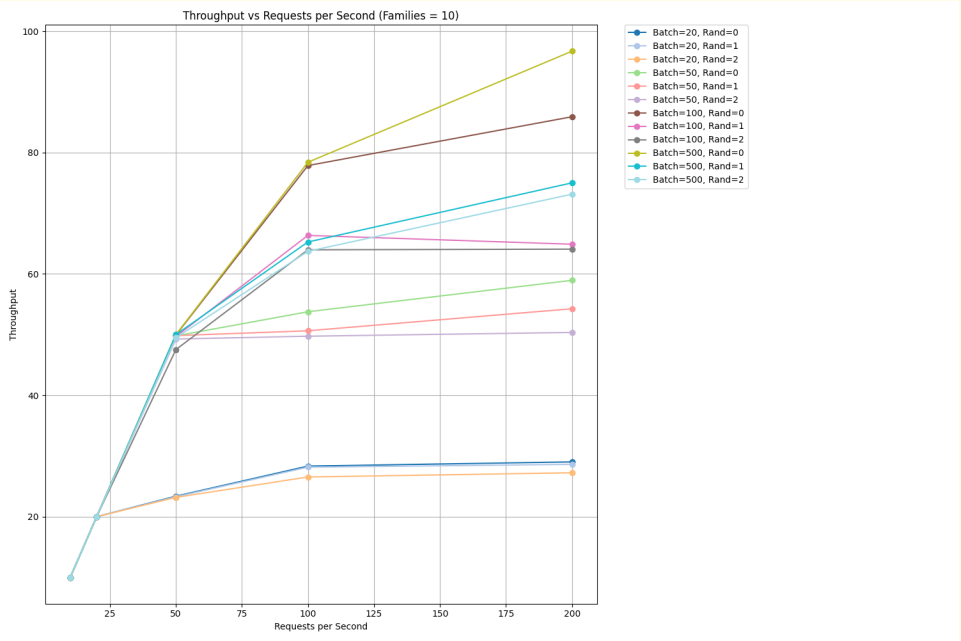


Figure: Throughput vs Workload by varying other parameters (10 families)

- Increase in batch size $\rightarrow$ higher throughput.
- A batch nature level of 1 is roughly similar to level 2 $\rightarrow$ even a small amount of mixing leads to significant degradation.
- A homogeneous batch of smaller size at times performs siginificantly better than a larger heterogeneous batch.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Varying Multiple Parameters Together

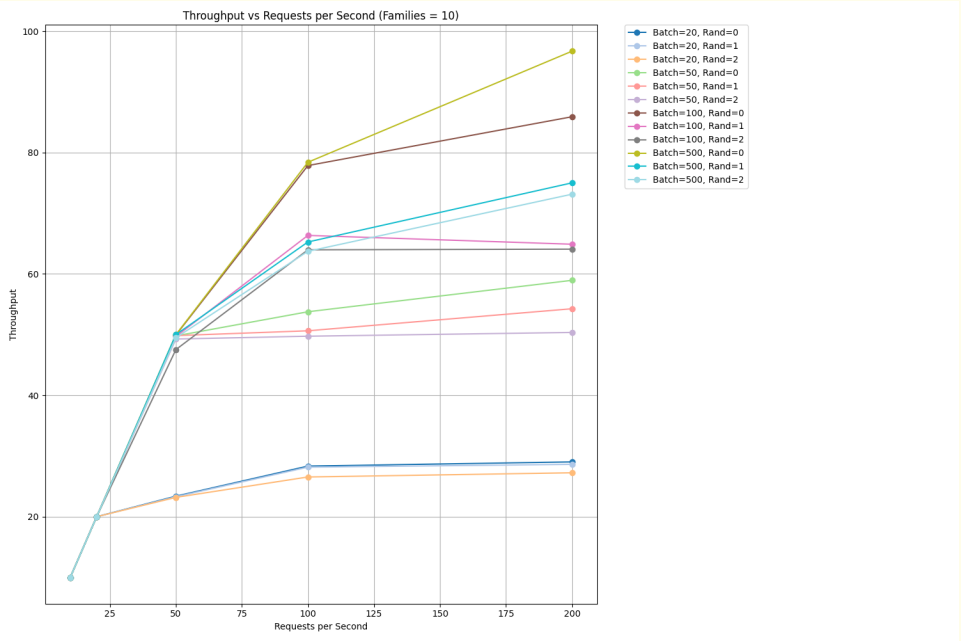


Figure: Throughput vs Workload by varying other parameters (10 families)

- Increase in batch size $\rightarrow$ higher throughput.
- A batch nature level of 1 is roughly similar to level 2 $\rightarrow$ even a small amount of mixing leads to significant degradation.
- A homogeneous batch of smaller size at times performs siginificantly better than a larger heterogeneous batch.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

Varying Chunk Size while Permuting

Table: Throughput across different k levels

k	**Throughput (req/sec)**
200	89.35
300	87.49
400	86.50
499	88.19
500	112.66
1000	112.41

- Before passing the requests to the model, we make chunks of size k, and permute each chunk randomly.
- A k value of 499 essentially means that in the first batch we have 499 requests of one family and 1 request of another family
- However the throughput is much lower than a k value of 500.
- Even one request of a different family will cause L2 cache thrashing.

DRAM Bandwidth Utilization - for Multiple Families

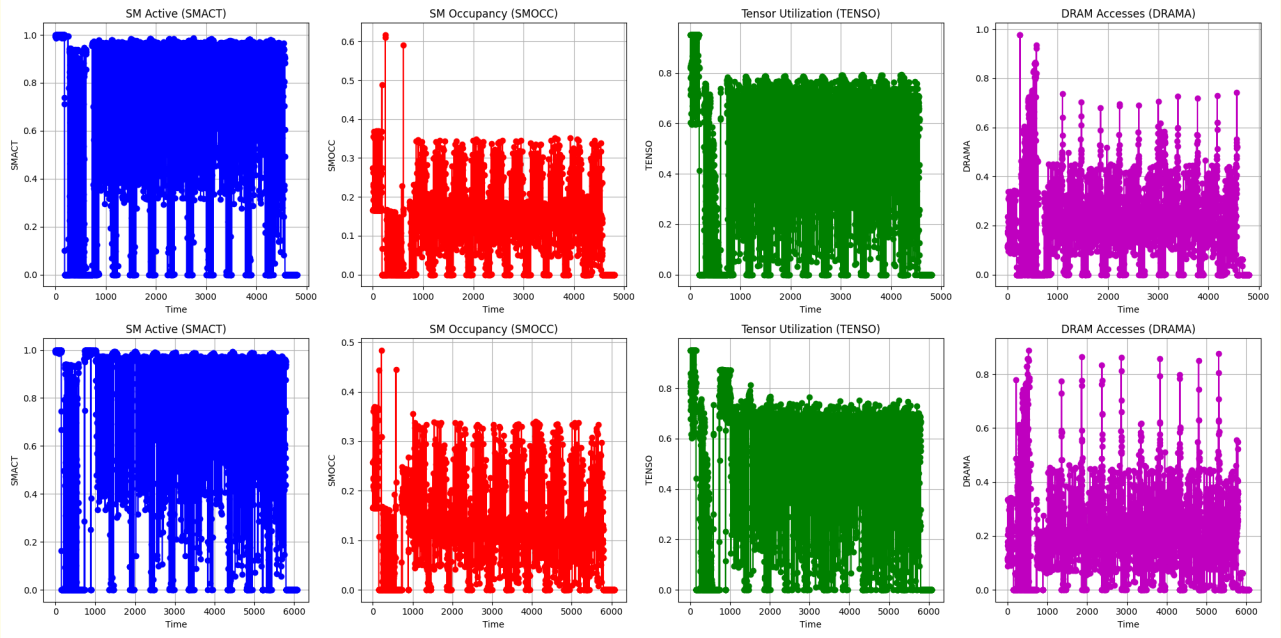


Figure: Comparison for $k = 500$ and $k = 499$

- L2 cache hit/miss rates $\rightarrow$ difficult to capture, as there are thousands of kernels forked.
- DRAMA - fraction of cycles where data was sent to or received from device memory.
- Average DRAMA for $k = 500 \rightarrow 0.18$; Average DRAMA for $k = 499 \rightarrow 0.22$.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

DRAM Bandwidth Utilization - for Multiple Families

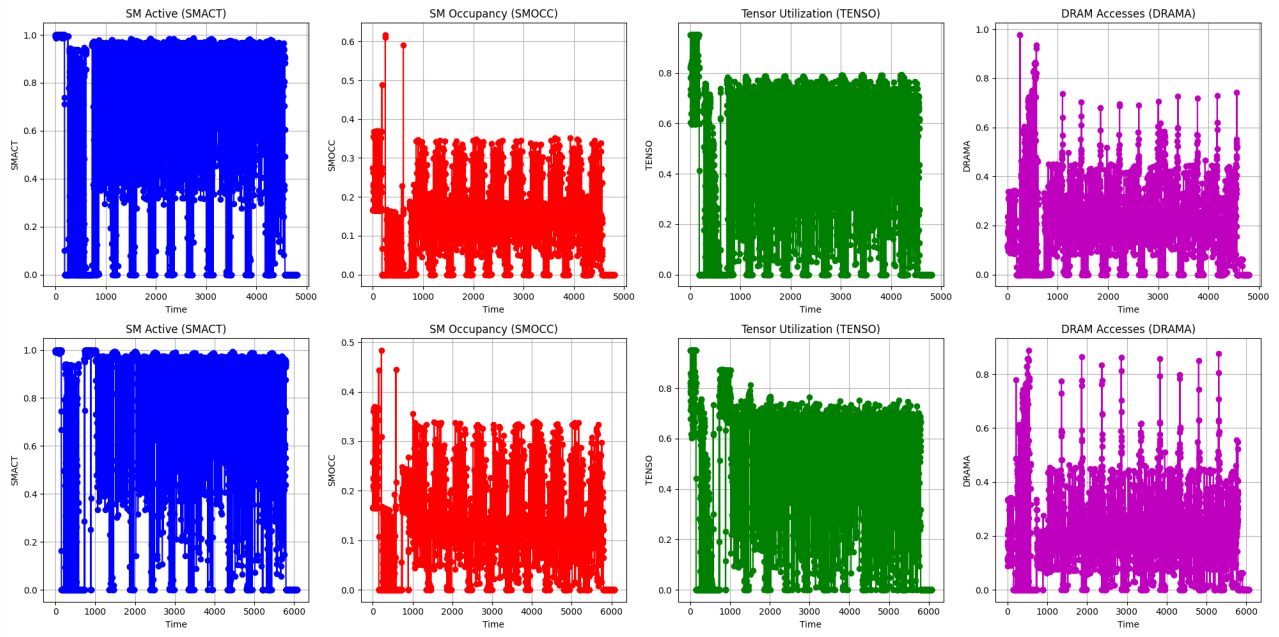


Figure: Comparison for $k = 500$ and $k = 499$

- L2 cache hit/miss rates $\rightarrow$ difficult to capture, as there are thousands of kernels forked.
- DRAMA - fraction of cycles where data was sent to or received from device memory.
- Average DRAMA for $k = 500 \rightarrow 0.18$; Average DRAMA for $k = 499 \rightarrow 0.22$.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

DRAM Bandwidth Utilization - for Multiple Families

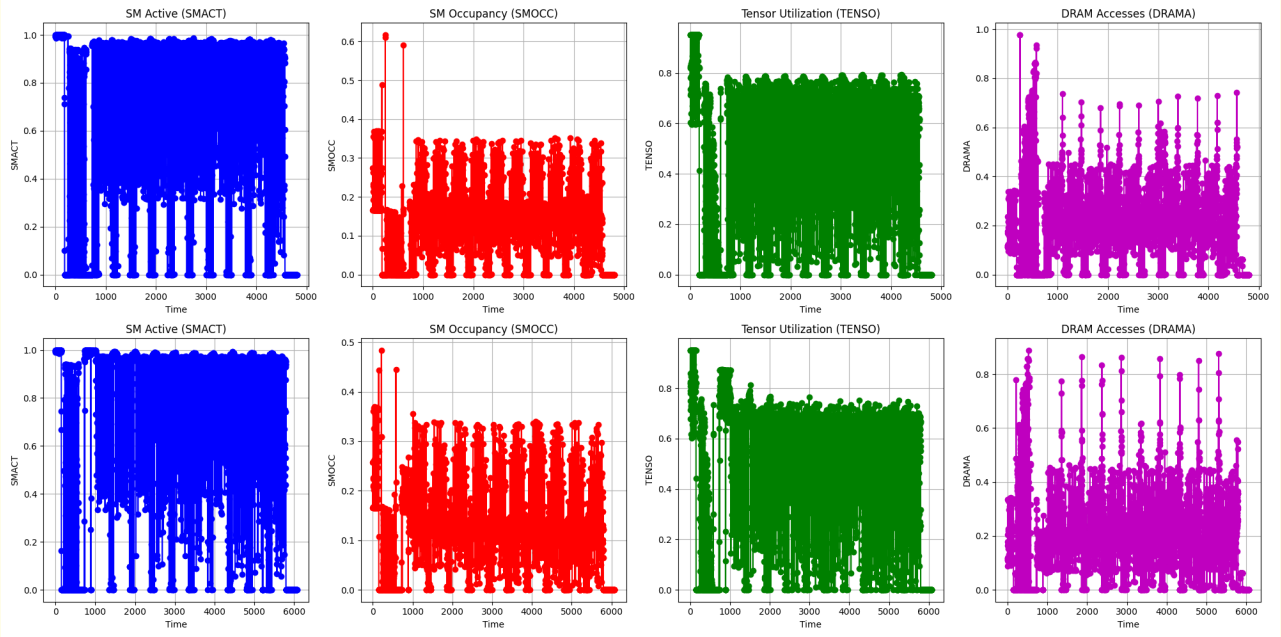


Figure: Comparison for $k = 500$ and $k = 499$

- L2 cache hit/miss rates $\rightarrow$ difficult to capture, as there are thousands of kernels forked.
- DRAMA - fraction of cycles where data was sent to or received from device memory.
- Average DRAMA for $k = 500 \rightarrow 0.18$; Average DRAMA for $k = 499 \rightarrow 0.22$.

Systems for LLMs
Saksham Rathi
Introduction
Background
Prior Work
Experiments - Single Family
Metrics
Experiments - Multiple Families
Future Steps

Future Steps



- **Improved Scheduler:** Design a scheduler that takes into account factors such as L2 cache size, the working set size of the current batch, the number and size of the KV cache blocks being targeted, and dynamically decides whether forming smaller homogeneous batches can outperform larger, mixed ones.
- **SM Level Optimization:** To increase the per-SM L1 cache hit rate, it is desirable to schedule requests from the same family on nearby (or the same) SMs. One possible approach is to assign each attention kernel an identifier associated derived from its family ID and preferentially schedule it on an SM already hosting requests from that family.

Future Steps



- **Improved Scheduler:** Design a scheduler that takes into account factors such as L2 cache size, the working set size of the current batch, the number and size of the KV cache blocks being targeted, and dynamically decides whether forming smaller homogeneous batches can outperform larger, mixed ones.
- **SM Level Optimization:** To increase the per-SM L1 cache hit rate, it is desirable to schedule requests from the same family on nearby (or the same) SMs. One possible approach is to assign each attention kernel an identifier associated derived from its family ID and preferentially schedule it on an SM already hosting requests from that family.

Future Steps



- **KV Cache Offloading:** As the number of families grows, the GPU's HBM may eventually become insufficient to hold all KV Cache blocks. Design an efficient eviction policy, ideally guided by the degree of prefix sharing among the currently scheduled batches.
- **Response Quality:** Another promising direction is to improve response quality for requests sharing a large common context. One approach is to fine-tune the model specifically for such shared contexts and compare its performance with that of a baseline model that receives the shared context as part of a system prompt.

Future Steps



- **KV Cache Offloading:** As the number of families grows, the GPU's HBM may eventually become insufficient to hold all KV Cache blocks. Design an efficient eviction policy, ideally guided by the degree of prefix sharing among the currently scheduled batches.
- **Response Quality:** Another promising direction is to improve response quality for requests sharing a large common context. One approach is to fine-tune the model specifically for such shared contexts and compare its performance with that of a baseline model that receives the shared context as part of a system prompt.



Thank You